

# ■ JobIQ — Master 50 Technical Interview Questions & Answers

**Complete Engineering Preparation Guide** for JobIQ Platform Architecture, FastAPI Backend, React 18 Frontend, PostgreSQL/Supabase Database, TF-IDF Machine Learning, Web Scrapers, Speech API, and Cloud DevOps.

---

## ■■ Section 1: Full-Stack Architecture & System Design

**Q1: Can you describe the end-to-end data flow when a candidate applies for a job on JobIQ?**

■ **Answer:** 1. **User Action:** The candidate clicks "Add to Pipeline" on an AI Match card in the React frontend. 2. **API Request:** React triggers an HTTP `POST /api/v1/applications/` request with the `opportunity_id` and the user's JWT token in the `Authorization: Bearer` header. 3. **Authentication Middleware:** FastAPI intercepts the request via `get_current_user`, validates the JWT token using `python-jose`, and extracts the `user_id`. 4. **ORM Execution:** SQLAlchemy creates an `Application` record with `status = "applied"` and `applied_at = datetime.utcnow()` in Supabase PostgreSQL. 5. **UI Update:** The frontend Zustand store updates local state instantly, reflecting the "Applied" badge on the card without needing a full page refresh.

**Q2: Why did you choose a Decoupled Monorepo structure over Microservices or a Monolith?**

■ **Answer:** A decoupled monorepo (`jobiq-backend/` + `jobiq-frontend/`) gives us the best of both worlds: - **Independent Scaling & Hosting:** The React frontend is deployed on Vercel's global Edge CDN, while the FastAPI backend runs on Render's Python container. They build and deploy independently. - **Single Source of Truth:** Having both repositories in one folder simplifies version control, shared documentation, and local developer setup (`npm run dev` + `uvicorn`). - **Low Overhead:** A microservices architecture would introduce unnecessary gRPC/RPC network latency and complex Kubernetes orchestration for an early-stage startup platform.

**Q3: How does JobIQ ensure sub-second response times across the platform?**

■ **Answer:** - **Database Indexing:** We created B-tree indexes on `opportunities.is_active`, `users.email`, and `applications.user_id`. - **Lightweight ML:** Matching uses sparse matrix TF-IDF vectorization with `scikit-learn` instead of blocking external LLM API calls, computing scores in under 15ms. - **Asynchronous Startup Seeding:** Scraping and DB bootstrapping are offloaded to background threads (`threading.Thread`) so FastAPI's Uvicorn server binds the web port in 0.1 seconds. - **Edge Static Delivery:** Frontend JavaScript/CSS bundles are cached at Vercel edge locations worldwide.

**Q4: How do you handle failure when an external job source (like LinkedIn or Naukri) goes down?**

■ **Answer:** Each scraper (`LinkedInScraper`, `NaukriScraper`, `InternshalaScraper`, `UnstopScraper`) inherits from `BaseScraper` and wraps HTTP calls in explicit `try/except` exception blocks. If a scraper encounters HTTP 429 (Rate Limit) or 503 (Service Unavailable), it logs the error silently and returns an empty list `[]` without crashing the overall ingestion pipeline.

**Q5: What design patterns did you use in the JobIQ codebase?**

■ **Answer:** - **Factory Pattern:** Used in `app/main.py` (`create_app()`) to instantiate FastAPI application instances. - **Template Method Pattern:** Implemented in `BaseScraper` where child scrapers override `fetch_opportunities()`. - **Repository/ORM Pattern:** Managed via SQLAlchemy `SessionLocal` for abstraction over SQL queries. - **Store Pattern (State Management):** Managed on the frontend via Zustand (`useAuthStore`, `useApplicationStore`). ---

---

## ■ Section 2: FastAPI & Async Backend Architecture

**Q6: Why FastAPI instead of Django or Flask?**

■ **Answer:** - **Speed:** FastAPI is built on Starlette and Pydantic, making it one of the fastest Python frameworks available (comparable to NodeJS and Go). - **Native OpenAPI/Swagger:** Automatically generates interactive API documentation at `/docs`. - **Data Validation:** Pydantic models automatically validate incoming JSON bodies and query parameters, returning standard 422 errors for malformed requests.

**Q7: What is Pydantic v2 and how is it used in JobIQ?**

■ **Answer:** Pydantic v2 provides strict data parsing and validation. In JobIQ, we separate database models from API DTOs: - `JobMatchRead`: Defines what fields are returned when reading an AI job match (joining opportunity title, company, location). - `UserCreate` & `UserRead`: Validates user registration payloads and strips out `hashed_password` before sending user data back to the client.

**Q8: How do background threads work in `app/main.py` during app startup?**

■ **Answer:** During the FastAPI `@app.on_event("startup")` hook, we run table creation synchronously (`Base.metadata.create_all`) and launch data bootstrapping in a background daemon thread: `python threading.Thread(target=bootstrap_data, daemon=True).start()` `Setting daemon=True` ensures the thread terminates automatically when the main process shuts down, preventing background worker leaks.

**Q9: How do Dependency Injection (`Depends`) work in FastAPI?**

■ **Answer:** FastAPI's `Depends` resolves dependencies before route execution. For example: `python def get_db(): db = SessionLocal() try: yield db finally: db.close()` `Injecting db: Session = Depends(get_db)` ensures every HTTP request receives an isolated DB session and automatically closes it when the response completes, preventing connection pool leaks.

**Q10: How do you handle global error handling and 404/500 responses in FastAPI?**

■ **Answer:** We use FastAPI's `HTTPException(status_code=404, detail="Opportunity not found")` for expected business logic errors. For unhandled exceptions, FastAPI automatically catches them and returns a JSON payload `{"detail": "Internal Server Error"}` with status 500, preventing server crashes. ---

---

## ■ Section 3: React 18, Vite & Frontend Engineering

### Q11: Why Vite instead of Create React App (CRA)?

■ **Answer:** - **Speed:** Vite uses native ES modules (ESM) in development and `esbuild` written in Go, starting the dev server in <300ms compared to CRA's Webpack which takes 20+ seconds. - **Production Build:** Vite uses Rollup for tree-shaking and chunk splitting, resulting in smaller static bundles.

### Q12: How does Zustand compare to Redux for state management in JobIQ?

■ **Answer:** Zustand provides a lightweight (1kB), boilerplate-free state store. Unlike Redux which requires actions, reducers, and dispatchers, Zustand allows direct hook consumption (`const { user } = useAuthStore()`) without wrapping the application in context providers.

### Q13: How do you protect private routes in React Router v6?

■ **Answer:** We wrap protected pages in a custom layout or route wrapper that checks `isAuthenticated` from `useAuthStore()`. If `false` or if no JWT token exists in `localStorage`, the user is automatically redirected to `/login` via .

### Q14: How does Axios handle automatic JWT token attachment?

■ **Answer:** In `src/api/client.ts`, we created an Axios request interceptor: ``typescript  
`apiClient.interceptors.request.use((config) => { const token = localStorage.getItem('jobiq_token'); if (token) config.headers.Authorization = Bearer ${token}; return config; });` `` Every outgoing API call automatically attaches the Bearer token without manual code repetition.

### Q15: What is Tailwind CSS glassmorphic UI design and how is it implemented?

■ **Answer:** We use utility classes like `backdrop-blur-md bg-slate-900/80 border border-slate-800/80` to create modern translucent card panels with glowing gradients (`bg-gradient-to-r from-indigo-500 to-purple-600`), elevating visual appeal. ---

---

## ■ Section 4: PostgreSQL, SQLAlchemy 2.0 & Supabase Database

### Q16: Explain your database schema design for JobIQ.

■ **Answer:** Our PostgreSQL schema consists of 6 primary tables: - `users`: `id`, `email`, `hashed_password`, `full_name`, `role`, `target_role`, `experience_level`. - `opportunities`: `id`, `title`, `company`, `location`, `salary_min`, `salary_max`, `required_skills` (JSONB), `source_url`, `source`, `is_active`. - `job_matches`: `id`, `user_id` (FK), `opportunity_id` (FK), `score`, `matching_skills`, `missing_skills`. - `applications`: `id`, `user_id` (FK), `opportunity_id` (FK), `status`, `applied_at`. - `resumes`: `id`, `user_id` (FK), `raw_text`, `parsed_skills`. - `saved_opportunities`: `id`, `user_id` (FK), `opportunity_id` (FK).

**Q17: What is SQLAlchemy 2.0 ORM and how does it differ from 1.x?**

■ **Answer:** SQLAlchemy 2.0 unifies the ORM and Core query syntaxes using explicit `select(Opportunity).where(...)` statements, provides strict typing compatibility with Python type hints, and handles execution pooling cleanly.

**Q18: What is connection pooling and why is `pool_pre_ping=True` critical?**

■ **Answer:** Connection pooling reuses database connections instead of opening a new TCP socket for every query. Setting `pool_pre_ping=True` sends a lightweight `SELECT 1` ping test before issuing queries to test if a stale connection was closed by Supabase/cloud firewall, preventing `ConnectionClosedError`.

**Q19: How do you handle JSON array columns in PostgreSQL using SQLAlchemy?**

■ **Answer:** We store required skills as JSON arrays (JSON type in SQLAlchemy). When filtering jobs by skill, we cast the JSON column to string in SQLAlchemy queries: `python query = query.filter(cast(Opportunity.required_skills, String).ilike(f"%{skill}%"))` This enables flexible case-insensitive substring searching across JSON lists.

**Q20: How do Foreign Key constraints (`ForeignKey`) protect data integrity?**

■ **Answer:** In `job_matches` and `applications`, `user_id` links to `users.id` and `opportunity_id` links to `opportunities.id`. Cascading deletes (`ondelete="CASCADE"`) ensure that if a user deletes their account, all associated match recommendations and application pipeline records are cleaned up automatically. ---

---

## ■ Section 5: Machine Learning, TF-IDF & Skill Extraction Engine

**Q21: Why use TF-IDF instead of BERT or OpenAI Embeddings for job matching?**

■ **Answer:** - **Latency:** TF-IDF runs in <15ms locally without network hops, whereas OpenAI API calls take 500-1500ms. - **Cost:** TF-IDF is 100% free with zero API costs. - **Privacy & Reliability:** Runs locally on the server without sending user resume data to third-party AI APIs.

**Q22: How does `TfidfVectorizer` work mathematically?**

■ **Answer:** TF-IDF measures how important a word is to a document within a collection: 
$$\text{TF}(t, d) = \frac{\text{Count of term } t \text{ in doc } d}{\text{Total terms in doc } d}$$
$$\text{IDF}(t, D) = \log\left(\frac{\text{Total documents } |D|}{1 + \text{Documents containing term } t}\right)$$
$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

**Q23: How is Cosine Similarity computed between two vectors?**

■ **Answer:** Cosine similarity measures the angle between two multi-dimensional vectors  $A$  and  $B$ :

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

A score of 1.0 means identical word frequencies; 0.0 means completely orthogonal (no word overlap).

**Q24: How does the Skill Extractor regex algorithm work?**

■ **Answer:** In `app/ml/skill_extractor.py`, we maintain a taxonomy of 200+ technology keywords. For each skill, we construct a word-boundary regex pattern: `python pattern = r'\b' + re.escape(skill.lower()) + r'\b'`` `word boundaries`` ensure that searching for "Java" does not accidentally match "JavaScript", and searching for "C" does not match "CSS".

**Q25: How does the Regex Resume Parser extract sections?**

■ **Answer:** In `app/ml/resume_parser.py`, we extract sections using multi-line regular expressions with `re.DOTALL`: - **Email:** `r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'` - **Phone:** `r'[\+]?[1-9]?[\d\s\-\(\)]{7,15}'` - **Sections:** Looks for headers like `EXPERIENCE:`, `EDUCATION:`, `PROJECTS:` and captures text up to the next structural header. ---

---

## ■ Section 6: Web Scraping, BeautifulSoup & Fuzzy Deduplication

**Q26: How does the LinkedIn Guest Scraper function without authentication?**

■ **Answer:** LinkedIn provides a public guest jobs API endpoint: <https://www.linkedin.com/jobs-guest/jobs/api/seeMoreJobPostings/search?keywords=Software+Engineer&location;=India&start;=0> We send an `httpx` GET request with a custom `User-Agent` header (`JobIQ/0.1`), parse the HTML response using `BeautifulSoup`, and extract `a.base-card__full-link` to get actual, clickable `linkedin.com/jobs/view/XXXX` links.

**Q27: How does `difflib.SequenceMatcher`` calculate string similarity?**

■ **Answer:** `SequenceMatcher` uses the Ratcliff/Obershelp algorithm to find the longest contiguous matching subsequence between two strings:  $\text{Ratio} = \frac{2 \cdot M}{T}$  where  $M$  is the number of matching characters and  $T$  is the total length of both strings combined.

**Q28: Explain the combined fuzzy score formula in `app/scrapers/deduplicator.py``.**

■ **Answer:** To determine if two opportunities are duplicates: `python title_sim = similarity(opp1.title, opp2.title) company_sim = similarity(opp1.company, opp2.company) combined_score = (title_sim * 0.6) + (company_sim * 0.4)`` Title similarity is weighted at 60% and company name similarity at 40%. If `combined_score >= 0.85`, `opp.is_active` is set to `False``.

**Q29: How do you respect rate limits when scraping external sites?**

■ **Answer:** `BaseScraper` includes a `_rate_limit()` helper that executes `time.sleep(2.0)` between paginated requests. This prevents IP bans and ensures compliance with external server load limits.

**Q30: How do you normalize salary and stipend data from raw scraped strings?**

■ **Answer:** Raw text like "■ 8,00,000 - ■ 15,00,000 P.A." is sanitized using regex `re.sub(r'[^\\d]', '', text)` to extract pure numeric digits, which are then converted to integer values in INR. ---

---

## ■ Section 7: AI Mock Interviewer & Speech API

**Q31: How does Web Speech API integration work in `InterviewPrepPage.tsx`?**

■ **Answer:** We initialize `window.SpeechRecognition` or `window.webkitSpeechRecognition`:  

```
``typescript const recognition = new window.webkitSpeechRecognition();
recognition.continuous = true; recognition.interimResults = true; recognition.onresult
= (event) => { const transcript = Array.from(event.results).map(r =>
r[0].transcript).join(''); setAnswer(transcript); }; ``
```

 This streams real-time speech transcription directly into local React state.

**Q32: How is the 300-question bank structured in `interviewQuestions.ts`?**

■ **Answer:** Each question object implements the TypeScript `Question` interface: 

```
``typescript export
interface Question { id: string; category: string; // e.g. "dbms", "oops", "sql"
title: string; tips: string; keywords: string[]; ideal_points: string[]; } ``
```

 There are exactly 25 questions per category across 12 categories: DBMS, OOPs, Computer Networks, Software Eng, Git, SQL, DSA, HR, Backend, Frontend, Python, and Java.

**Q33: How does real-time keyword detection work as the candidate speaks?**

■ **Answer:** As `answer` text updates in state, we evaluate matching keywords: 

```
``typescript const matched =
question.keywords.filter(kw => answer.toLowerCase().includes(kw.toLowerCase()));
const score = Math.round((matched.length / question.keywords.length) * 100); ``
```

 Matching keywords are highlighted green on screen with live checkmarks.

**Q34: How is speech synthesis (Text-to-Speech) implemented?**

■ **Answer:** We use `window.speechSynthesis.speak(new SpeechSynthesisUtterance(question.title))` to allow the AI interviewer to read out question titles aloud.

**Q35: How do you handle browser compatibility for speech recognition?**

■ **Answer:** We check if `webkitSpeechRecognition` is supported before mounting the microphone control:

```
``typescript const isSpeechSupported = 'SpeechRecognition' in window ||  
'webkitSpeechRecognition' in window; ``
```

If unsupported (e.g. Firefox), the UI displays a fallback text area for manual typing. ---

---

## ■ Section 8: Authentication, Security & CORS

**Q36: What is a JWT token and what are its three parts?**

■ **Answer:** A JWT (JSON Web Token) consists of three dot-separated base64-encoded strings: 1. **Header:** Algorithm (`HS256`) and token type (`JWT`). 2. **Payload:** Claims containing `sub` (user\_id) and `exp` (expiration timestamp). 3. **Signature:** Cryptographic signature created using `HMAC-SHA256(header + payload, secret_key)`.

**Q37: Why use `bcrypt` for password hashing instead of `SHA256` or `MD5`?**

■ **Answer:** `SHA256` and `MD5` are fast hash functions, making them vulnerable to brute-force and rainbow table attacks. `bcrypt` incorporates an adaptive work factor (cost parameter) and automatic random salt generation, making password hashing intentionally computational-heavy and resistant to GPU cracking.

**Q38: How do you prevent SQL Injection in JobIQ?**

■ **Answer:** SQLAlchemy ORM automatically uses parameterized queries (prepared statements). Input values are never concatenated directly into SQL strings, preventing malicious payloads (like `' OR 1=1 --`) from altering SQL query execution logic.

**Q39: How do you prevent XSS (Cross-Site Scripting) attacks in React?**

■ **Answer:** React automatically escapes values embedded in JSX string expressions before rendering them to the DOM, preventing script injection.

**Q40: How is sensitive data (DB passwords, secret keys) secured in production?**

■ **Answer:** Sensitive keys are stored strictly in environment variables (`DATABASE_URL`, `SECRET_KEY`) managed via Render/Vercel dashboard settings, never committed to git repositories (`.gitignore` includes `.env`). ---

---

## ■ Section 9: Application Pipeline & State Management



**Q41: Describe the application status state machine in JobIQ.**

■ **Answer:** An application transitions through discrete states: `Saved`  $\rightarrow$  `Applied`  $\rightarrow$  `Interviewing`  $\rightarrow$  `Offered / Rejected` The frontend Kanban view categorizes applications into distinct visual columns based on `status`.

**Q42: How does optimistic UI updating improve user experience?**

■ **Answer:** When a user clicks "Add to Pipeline", the Zustand store immediately adds the application to local UI state and shows an "Added" confirmation badge before the HTTP POST network request finishes. If the server request fails, the store rolls back the optimistic update and displays an alert.

**Q43: How do you prevent duplicate job applications for the same user?**

■ **Answer:** Before creating an `Application` record, `app/routes/applications.py` queries: `python existing = db.query(Application).filter( Application.user_id == current_user.id, Application.opportunity_id == opp_id ).first() `` If an application already exists, the API returns a 400 Bad Request ("Application already exists").

**Q44: How does saved opportunities toggle work in Zustand?**

■ **Answer:** `useSavedOpportunitiesStore` maintains an array of saved `opportunity_ids` in `localStorage`. Toggling a bookmark icon instantly updates `localStorage` and synchronized badge counts.

**Q45: How do you structure TypeScript types for API responses?**

■ **Answer:** In `src/types/index.ts`, we define strict interfaces matching backend schemas: `typescript export interface Opportunity { id: number; title: string; company: string; location: string | null; salary_min: number | null; salary_max: number | null; required_skills: string[]; source_url: string; source: string; type: string; } ``---

---

## ■ Section 10: DevOps, Vercel, Render & Production Troubleshooting

**Q46: Explain the difference between Vercel and Render for hosting.**

■ **Answer:** - **Vercel:** Optimized for static site generation (SSG), single-page apps (SPA), and Edge CDN delivery. Hosts our React frontend. - **Render:** Docker and Python container hosting platform. Manages our persistent FastAPI Uvicorn web service process.

**Q47: What is CI/CD and how is it configured for JobIQ?**

■ **Answer:** Continuous Integration / Continuous Deployment: Whenever code is committed and pushed to `main` on GitHub (`git push origin main`): 1. **Vercel Webhook:** Triggers an automatic build (`tsc -b && vite build`) and deploys static assets to global edge locations in 20 seconds. 2. **Render Webhook:** Pulls the latest code, installs Python wheels (`pip install -r requirements.txt`), and restarts Uvicorn server automatically.



#### Q48: How did you fix the `No open ports detected` Render deployment error?

■ **Answer:** During container startup, Render scans for open HTTP ports. Initially, synchronous scraping calls inside `@app.on_event("startup")` blocked Uvicorn from binding port `$PORT` within 30 seconds. **Fix:** We moved `bootstrap_data()` into a background daemon thread (`threading.Thread`). FastAPI opens port `$PORT` in 0.1 seconds, and Render scans pass immediately (`Status: Live` ■).

#### Q49: How did unpinning `scikit-learn` in `requirements.txt` speed up Render builds?

■ **Answer:** Pinning an exact C-extension version (`scikit-learn==1.6.1`) forced Render to download raw C++ source files (`.tar.gz`) and compile them, taking 10+ minutes. Unpinning (`scikit-learn>=1.3.0`) allowed `pip` to download pre-compiled binary wheels (`.whl`), reducing build time from 10 minutes to **8 seconds**.

#### Q50: What are your future architectural improvements for JobIQ?

■ **Answer:** 1. **Redis Caching:** Cache popular job search queries in Redis to reduce PostgreSQL read load. 2. **Vector Embeddings (pgvector):** Migrate from TF-IDF to semantic vector embeddings using OpenAI/SentenceTransformers stored in PostgreSQL via `pgvector`. 3. **Celery Distributed Workers:** Run web scrapers on scheduled Celery cron workers using Redis as message broker. --- \*Document saved as `JobIQ_50_Technical_Interview_Questions.md`.\*